
TP N°1 : Rappels SQL sous MySQL

Objectifs

Ce premier TP remet en pratique les notions revues au début du module. À la fin de la séance, vous devez être capables de créer une base MySQL avec ses contraintes d'intégrité, d'en faire évoluer la structure, d'y insérer et modifier des données, puis de l'interroger avec des jointures, des regroupements, des sous-requêtes et des opérateurs ensemblistes.

Consignes

Tout le travail est rendu dans un seul script nommé TP1_Nom.sql, déposé sur Moodle à la fin de la séance. Avant chaque requête, indiquez en commentaire le numéro de la question (par exemple -- 3.7). Les réponses aux questions de réflexion sont aussi écrites en commentaire.

Le script doit pouvoir être rejoué du début à la fin sur une base vide sans s'arrêter sur une erreur. Le fichier tp1_donnees.sql vous est fourni pour remplir la base à partir de l'exercice 2. La qualité du script (commentaires, exécution sans erreur) compte pour 1 point.

Présentation du cas : AtlasCar

AtlasCar est un loueur de véhicules implanté dans la région Souss-Massa. Ses données sont aujourd'hui éparpillées dans plusieurs fichiers Excel et le gérant souhaite passer à une base MySQL. Les règles de gestion retenues sont les suivantes :

- RG1** AtlasCar possède plusieurs agences. Chaque agence a un numéro, un nom, une ville et un numéro de téléphone qui lui est propre.
- RG2** Un véhicule est identifié par son immatriculation. On connaît sa marque, son modèle, son année de mise en circulation, son kilométrage (0 pour un véhicule neuf, jamais négatif) et son carburant (Essence, Diesel, Hybride ou Electrique).
- RG3** Un véhicule est rattaché à une seule agence. Une agence peut n'avoir encore aucun véhicule (agence en cours d'ouverture).
- RG4** Chaque véhicule appartient à une catégorie (Economique, Citadine, SUV...) identifiée par un code de 3 lettres. La catégorie fixe le tarif journalier, qui est strictement positif.
- RG5** Un client est identifié par un numéro interne. On enregistre aussi son CIN (obligatoire et unique), son nom, son prénom, sa ville, son téléphone et la date d'obtention de son permis.
- RG6** Une location concerne un client et un véhicule. Elle a une date de début, une date de fin (vide tant que le véhicule n'est pas rendu), le kilométrage au départ et au retour. La date de fin ne peut pas précéder la date de début.
- RG7** Une location peut inclure des équipements (GPS, siège bébé, galerie de toit...) facturés à la journée. On note pour chaque location la quantité de chaque équipement, au moins 1.
- RG8** Le montant d'une location n'est pas stocké : il se calcule à partir de la durée, du tarif de la catégorie et des équipements.

Schéma relationnel

Les clés primaires sont soulignées et les clés étrangères précédées de #. Les noms des tables et des colonnes doivent être respectés à l'identique, le fichier de données en dépend.

Agence (id_agence, nom_agence, ville, telephone)

Categorie (code_cat, libelle, tarif_jour)

Vehicule (immatriculation, marque, modele, annee, kilometrage, carburant, #code_cat, #id_agence)

Client (id_client, cin, nom, prenom, ville, telephone, date_permis)

Location (id_location, date_debut, date_fin, km_depart, km_retour, #id_client, #immatriculation)

Equipement (code_equip, libelle, prix_jour)

Location_Equipement (#id_location, #code_equip, quantite)

Remarque : OPTION étant un mot réservé en MySQL, la table des options s'appelle Equipement.

Exercice 1 : Création de la base (LDD)

Toutes les contraintes doivent être nommées (CONSTRAINT nom_contrainte ...) afin de pouvoir les modifier ou les supprimer plus tard.

- 1.1 Créez la base atlascar puis sélectionnez-la. Placez un DROP DATABASE IF EXISTS atlascar; en tête de script pour pouvoir le relancer.
- 1.2 Dans quel ordre faut-il créer les tables ? Justifiez votre réponse.
- 1.3 Créez les sept tables en respectant les contraintes du tableau ci-dessous. Choisissez des types adaptés (INT, SMALLINT, VARCHAR(n), CHAR(n), DECIMAL(p,s), DATE) et justifiez en commentaire le choix du type de code_cat et de tarif_jour.

Table	Contraintes à mettre en place
Agence	clé primaire auto-incrémentée ; nom et ville obligatoires ; téléphone unique
Categorie	clé primaire sur un code de 3 caractères ; tarif obligatoire et > 0
Vehicule	kilométrage ≥ 0, égal à 0 par défaut ; carburant limité aux 4 valeurs de RG2 ; catégorie et agence obligatoires
Client	clé primaire auto-incrémentée ; CIN obligatoire et unique ; nom et prénom obligatoires
Location	date de début obligatoire ; date de fin facultative mais ≥ date de début ; client et véhicule obligatoires
Equipement	libellé obligatoire ; prix journalier > 0
Location_Equipement	clé primaire composée ; quantité ≥ 1, égale à 1 par défaut

1.4 Modifiez la structure avec ALTER TABLE :

- a) ajoutez une colonne email à la table Client, puis une contrainte d'unicité nommée uq_client_email ;
- b) ajoutez à Location la contrainte ck_loc_km : le kilométrage de retour, s'il est connu, doit être supérieur ou égal à celui du départ ;
- c) ajoutez à Vehicule une colonne statut obligatoire, valant 'disponible' par défaut et limitée aux valeurs 'disponible' et 'maintenance' ;
- d) passez le téléphone des agences en VARCHAR(20) ;

- e) créez une table Brouillon (x INT) puis supprimez-la sans provoquer d'erreur si elle n'existe pas.

1.5 Vérifiez votre travail avec `SHOW TABLES`, `DESCRIBE Vehicule` et `SHOW CREATE TABLE Location`. Retrouvez-vous bien les noms de vos contraintes ?

N.B. : depuis la version 8.0.16, MySQL vérifie réellement les contraintes CHECK (les versions précédentes les acceptaient mais les ignoraient). Vérifiez votre version avec `SELECT VERSION()` ;.

Exercice 2 : Manipulation des données (LMD)

- 2.1** Exécutez le script `tp1_donnees.sql`, puis vérifiez le nombre de lignes de chaque table. Vous devez trouver 5 agences, 5 catégories, 12 véhicules, 10 clients, 19 locations, 5 équipements et 11 lignes dans `Location_Equipement`.
- 2.2** Pour chacun des ordres suivants, écrivez d'abord en commentaire le résultat attendu, puis exécutez-le et notez la contrainte qui a réagi. Pensez ensuite à mettre ces ordres en commentaire pour que votre script reste exécutable.
 - a) l'ajout d'un client dont le CIN est 'JB123456' ;
 - b) l'ajout d'un véhicule de catégorie 'XXX' ;
 - c) l'ajout d'un véhicule dont le kilométrage vaut -50 ;
 - d) l'ajout d'un véhicule qui roule au GPL ;
 - e) l'ajout d'une location qui commence le 10/10/2026 et se termine le 05/10/2026 ;
 - f) la suppression de l'agence n°1 ;
 - g) l'ajout d'un client sans CIN.
- 2.3** L'agence d'Inezgane reçoit une Renault Kardian 2026, citadine, essence, immatriculée '60001-A-40'. Ajoutez-la sans préciser le kilométrage ni le statut. Quelles valeurs obtenez-vous pour ces deux colonnes ?
- 2.4** Inscrivez-vous comme client sans donner de valeur à `id_client`. Quel numéro MySQL vous a-t-il attribué et pourquoi ?
- 2.5** Le tarif de la catégorie SUV augmente de 10 %. Faites la mise à jour en une seule requête.
- 2.6** Renseignez l'email de Youssef Alaoui (`y.alaoui@mail.ma`) et celui de Salma Benali (`s.benali@mail.ma`). Essayez ensuite d'attribuer l'email de Salma à un autre client. Que se passe-t-il ?
- 2.7** La Golf immatriculée '32010-B-40' est rendue le 27/09/2026 avec 54 100 km au compteur. Clôturez la location en cours correspondante, puis reportez ce kilométrage sur le véhicule.
- 2.8** La Hyundai Accent part en maintenance. Modifiez son statut en la désignant par sa marque et son modèle.
- 2.9** Supprimez l'équipement « Routeur Wi-Fi », qui n'a jamais été loué. Essayez ensuite de supprimer le GPS : expliquez l'erreur obtenue et ce qu'il faudrait faire pour y arriver.
- 2.10** Sans l'exécuter, expliquez ce que ferait l'ordre `DELETE FROM Location_Equipement`; et en quoi il diffère de `DROP TABLE Location_Equipement`;

À la fin de cet exercice, la base doit contenir 13 véhicules, 11 clients et 4 équipements.

Exercice 3 : Interrogation de la base

Pour tout l'exercice, on considère que la durée facturée d'une location est égale à `DATEDIFF(date_fin, date_debut)` jours.

A. Sélections simples

- 3.1 Donnez l'immatriculation, la marque et le modèle des véhicules hybrides ou électriques, triés par marque puis par modèle.
- 3.2 Affichez les clients dont le nom commence par la lettre A.
- 3.3 Affichez les véhicules mis en circulation entre 2023 et 2025 (inclus) qui ont moins de 30 000 km.
- 3.4 Affichez les locations en cours. Pourquoi la condition `date_fin = NULL` ne renvoie-t-elle aucune ligne ?
- 3.5 Donnez la liste des villes des clients, sans doublon et par ordre alphabétique.

B. Jointures

- 3.6 Pour chaque véhicule, affichez son immatriculation, sa marque, son modèle, le libellé et le tarif de sa catégorie ainsi que le nom de son agence.
- 3.7 Affichez les locations commencées en 2026 avec le nom et le prénom du client et la marque et le modèle du véhicule, triées par date de début.
- 3.8 Pour chaque location terminée, affichez le nombre de jours, le tarif journalier et le montant de base (nombre de jours $\times$ tarif), du plus cher au moins cher.
- 3.9 Affichez toutes les agences avec leurs véhicules, y compris les agences qui n'en ont aucun. Quelle agence apparaît avec des valeurs NULL ?
- 3.10 En utilisant une jointure externe, retrouvez les clients qui n'ont jamais loué de véhicule.

C. Fonctions de groupe

- 3.11 Donnez le nombre de véhicules par agence. Une agence sans véhicule doit afficher 0 et non 1.
- 3.12 Pour chaque catégorie, donnez le nombre de locations terminées et le total des kilomètres parcourus.
- 3.13 Calculez le chiffre d'affaires hors équipements réalisé avec chaque client, du plus élevé au plus faible.
- 3.14 Affichez les clients qui ont effectué au moins 3 locations.
- 3.15 Pour chaque carburant, donnez le kilométrage moyen, minimum et maximum, en ne gardant que les carburants dont la moyenne dépasse 20 000 km. Pourquoi ce filtre ne peut-il pas être placé dans le WHERE ?

D. Sous-requêtes

- 3.16 Affichez les véhicules dont le kilométrage est supérieur à la moyenne du parc.
- 3.17 Avec `NOT IN`, affichez les véhicules qui n'ont jamais été loués.
- 3.18 Affichez les clients ayant loué au moins un SUV, en utilisant uniquement des `IN` imbriqués (aucune jointure).
- 3.19 Avec `ALL`, affichez les véhicules moins kilométrés que tous les véhicules de l'agence Agadir Centre.
- 3.20 Avec `NOT EXISTS`, affichez les clients qui n'ont jamais loué de véhicule hybride.

E. Opérateurs ensemblistes

- 3.21 Affichez les villes où AtlasCar a une agence ou au moins un client, sans doublon. Combien de lignes obtenez-vous avec `UNION ALL` ? Expliquez.
- 3.22 Affichez les villes où AtlasCar a à la fois une agence et des clients, d'abord avec `INTERSECT` (MySQL 8.0.31 ou plus), puis avec une sous-requête `IN`.

Exercice 4 : Pour aller plus loin (bonus)

Ces questions sont réservées aux binômes qui ont terminé les exercices précédents.

- 4.1** Calculez le montant total de chaque location terminée, équipements compris : nombre de jours $\times$ (tarif de la catégorie + somme des prix journaliers $\times$ quantités). Une location sans équipement doit rester dans le résultat (pensez à LEFT JOIN et à COALESCE).
- 4.2** Pour chaque agence, affichez le ou les véhicules les plus loués. En cas d'égalité, tous doivent apparaître.
- 4.3** Retrouvez le client qui a le plus dépensé (hors équipements) sans utiliser LIMIT. Pourquoi une solution avec ORDER BY ... LIMIT 1 serait-elle fragile ?